ELF Header

Some object file control structures can grow, because the ELF header contains their actual sizes. If the object file format changes, a program may encounter control structures that are larger or smaller than expected. Programs might therefore ignore ``extra'' information. The treatment of ``missing'' information depends on context and will be specified when and if extensions are defined.

Figure 4-3: ELF Header

```
#define EI_NIDENT 16

typedef struct {
        unsigned char   e_ident[EI_NIDENT];
        Elf32_Half      e_type;
        Elf32_Half      e_machine;
        Elf32_Word      e_version;
        Elf32_Addr      e_entry;
        Elf32_Off       e_phoff;
        Elf32_Off       e_shoff;
        Elf32_Word      e_flags;
        Elf32_Half      e_ehsize;
        Elf32_Half      e_phentsize;
        Elf32_Half      e_phnum;
        Elf32_Half      e_shentsize;
        Elf32_Half      e_shnum;
        Elf32_Half      e_shstrndx;
} Elf32_Ehdr;

typedef struct {
        unsigned char   e_ident[EI_NIDENT];
        Elf64_Half      e_type;
        Elf64_Half      e_machine;
        Elf64_Word      e_version;
        Elf64_Addr      e_entry;
        Elf64_Off       e_phoff;
        Elf64_Off       e_shoff;
        Elf64_Word      e_flags;
        Elf64_Half      e_ehsize;
        Elf64_Half      e_phentsize;
        Elf64_Half      e_phnum;
        Elf64_Half      e_shentsize;
        Elf64_Half      e_shnum;
        Elf64_Half      e_shstrndx;
} Elf64_Ehdr;
```

`e_ident`
: The initial bytes mark the file as an object file and provide machine-independent data with which to decode and interpret the file's contents. Complete descriptions appear below in ``ELF Identification''.

`e_type`
: This member identifies the object file type.

Name	Value	Meaning
ET_NONE	0	No file type
ET_REL	1	Relocatable file
ET_EXEC	2	Executable file
ET_DYN	3	Shared object file

ET_CORE	4	Core file
ET_LOOS	0xfe00	Operating system-specific
ET_HIOS	0xfeff	Operating system-specific
ET_LOPROC	0xff00	Processor-specific
ET_HIPROC	0xffff	Processor-specific

Although the core file contents are unspecified, type ET_CORE is reserved to mark the file. Values from ET_LOOS through ET_HIOS (inclusive) are reserved for operating system-specific semantics. Values from ET_LOPROC through ET_HIPROC (inclusive) are reserved for processor-specific semantics. If meanings are specified, the processor supplement explains them. Other values are reserved and will be assigned to new object file types as necessary.

e_machine

This member's value specifies the required architecture for an individual file.

Name	Value	Meaning
EM_NONE	0	No machine
EM_M32	1	AT&T WE 32100
EM_SPARC	2	SPARC
EM_386	3	Intel 80386
EM_68K	4	Motorola 68000
EM_88K	5	Motorola 88000
reserved	6	Reserved for future use (was EM_486)
EM_860	7	Intel 80860
EM_MIPS	8	MIPS I Architecture
EM_S370	9	IBM System/370 Processor
EM_MIPS_RS3_LE	10	MIPS RS3000 Little-endian
reserved	11–14	Reserved for future use
EM_PARISC	15	Hewlett-Packard PA-RISC
reserved	16	Reserved for future use
EM_VPP500	17	Fujitsu VPP500
EM_SPARC32PLUS	18	Enhanced instruction set SPARC
EM_960	19	Intel 80960
EM_PPC	20	PowerPC
EM_PPC64	21	64-bit PowerPC
EM_S390	22	IBM System/390 Processor
reserved	23–35	Reserved for future use
EM_V800	36	NEC V800
EM_FR20	37	Fujitsu FR20
EM_RH32	38	TRW RH-32
EM_RCE	39	Motorola RCE
EM_ARM	40	Advanced RISC Machines ARM
EM_ALPHA	41	Digital Alpha
EM_SH	42	Hitachi SH
EM_SPARCV9	43	SPARC Version 9
EM_TRICORE	44	Siemens TriCore embedded processor

EM_ARC	45	Argonaut RISC Core, Argonaut Technologies Inc.
EM_H8_300	46	Hitachi H8/300
EM_H8_300H	47	Hitachi H8/300H
EM_H8S	48	Hitachi H8S
EM_H8_500	49	Hitachi H8/500
EM_IA_64	50	Intel IA-64 processor architecture
EM_MIPS_X	51	Stanford MIPS-X
EM_COLDFIRE	52	Motorola ColdFire
EM_68HC12	53	Motorola M68HC12
EM_MMA	54	Fujitsu MMA Multimedia Accelerator
EM_PCP	55	Siemens PCP
EM_NCPU	56	Sony nCPU embedded RISC processor
EM_NDR1	57	Denso NDR1 microprocessor
EM_STARCORE	58	Motorola Star*Core processor
EM_ME16	59	Toyota ME16 processor
EM_ST100	60	STMicroelectronics ST100 processor
EM_TINYJ	61	Advanced Logic Corp. TinyJ embedded processor family
EM_X86_64	62	AMD x86-64 architecture
EM_PDSP	63	Sony DSP Processor
EM_PDP10	64	Digital Equipment Corp. PDP-10
EM_PDP11	65	Digital Equipment Corp. PDP-11
EM_FX66	66	Siemens FX66 microcontroller
EM_ST9PLUS	67	STMicroelectronics ST9+ 8/16 bit microcontroller
EM_ST7	68	STMicroelectronics ST7 8-bit microcontroller
EM_68HC16	69	Motorola MC68HC16 Microcontroller
EM_68HC11	70	Motorola MC68HC11 Microcontroller
EM_68HC08	71	Motorola MC68HC08 Microcontroller
EM_68HC05	72	Motorola MC68HC05 Microcontroller
EM_SVX	73	Silicon Graphics SVx
EM_ST19	74	STMicroelectronics ST19 8-bit microcontroller
EM_VAX	75	Digital VAX
EM_CRIS	76	Axis Communications 32-bit embedded processor
EM_JAVELIN	77	Infineon Technologies 32-bit embedded processor
EM_FIREPATH	78	Element 14 64-bit DSP Processor
EM_ZSP	79	LSI Logic 16-bit DSP Processor
EM_MMIX	80	Donald Knuth's educational 64-bit processor
EM_HUANY	81	Harvard University machine-independent object files
EM_PRISM	82	SiTera Prism
EM_AVR	83	Atmel AVR 8-bit microcontroller
EM_FR30	84	Fujitsu FR30
EM_D10V	85	Mitsubishi D10V
EM_D30V	86	Mitsubishi D30V
EM_V850	87	NEC v850
EM_M32R	88	Mitsubishi M32R

EM_MN10300	89	Matsushita MN10300
EM_MN10200	90	Matsushita MN10200
EM_PJ	91	picoJava
EM_OPENRISC	92	OpenRISC 32-bit embedded processor
EM_ARC_A5	93	ARC Cores Tangent-A5
EM_XTENSA	94	Tensilica Xtensa Architecture
EM_VIDEOCORE	95	Alphamosaic VideoCore processor
EM_TMM_GPP	96	Thompson Multimedia General Purpose Processor
EM_NS32K	97	National Semiconductor 32000 series
EM_TPC	98	Tenor Network TPC processor
EM_SNP1K	99	Trebia SNP 1000 processor
EM_ST200	100	STMicroelectronics (www.st.com) ST200 microcontroller

Other values are reserved and will be assigned to new machines as necessary. Processor-specific ELF names use the machine name to distinguish them. For example, the flags mentioned below use the prefix EF_; a flag named WIDGET for the EM_XYZ machine would be called EF_XYZ_WIDGET.

e_version
: This member identifies the object file version.

Name	Value	Meaning
EV_NONE	0	Invalid version
EV_CURRENT	1	Current version

The value 1 signifies the original file format; extensions will create new versions with higher numbers. Although the value of EV_CURRENT is shown as 1 in the previous table, it will change as necessary to reflect the current version number.

e_entry
: This member gives the virtual address to which the system first transfers control, thus starting the process. If the file has no associated entry point, this member holds zero.

e_phoff
: This member holds the program header table's file offset in bytes. If the file has no program header table, this member holds zero.

e_shoff
: This member holds the section header table's file offset in bytes. If the file has no section header table, this member holds zero.

e_flags
: This member holds processor-specific flags associated with the file. Flag names take the form EF_*machine_flag*.

e_ehsize
: This member holds the ELF header's size in bytes.

e_phentsize
: This member holds the size in bytes of one entry in the file's program header table; all entries are the same size.

e_phnum
: This member holds the number of entries in the program header table. Thus the product of e_phentsize and e_phnum gives the table's size in bytes. If a file has no program header table, e_phnum holds the value zero.

e_shentsize
: This member holds a section header's size in bytes. A section header is one entry in the section header table; all entries are the same size.

e_shnum

This member holds the number of entries in the section header table. Thus the product of `e_shentsize` and `e_shnum` gives the section header table's size in bytes. If a file has no section header table, `e_shnum` holds the value zero.

If the number of sections is greater than or equal to `SHN_LORESERVE` (`0xff00`), this member has the value zero and the actual number of section header table entries is contained in the `sh_size` field of the section header at index `0`. (Otherwise, the `sh_size` member of the initial entry contains `0`.)

`e_shstrndx`
This member holds the section header table index of the entry associated with the section name string table. If the file has no section name string table, this member holds the value `SHN_UNDEF`. See ``Sections'' and ``String Table'' below for more information.

If the section name string table section index is greater than or equal to `SHN_LORESERVE` (`0xff00`), this member has the value `SHN_XINDEX` (`0xffff`) and the actual index of the section name string table section is contained in the `sh_link` field of the section header at index `0`. (Otherwise, the `sh_link` member of the initial entry contains `0`.)

ELF Identification

As mentioned above, ELF provides an object file framework to support multiple processors, multiple data encodings, and multiple classes of machines. To support this object file family, the initial bytes of the file specify how to interpret the file, independent of the processor on which the inquiry is made and independent of the file's remaining contents.

The initial bytes of an ELF header (and an object file) correspond to the `e_ident` member.

Figure 4-4: `e_ident[]` **Identification Indexes**

Name	Value	Purpose
`EI_MAG0`	`0`	File identification
`EI_MAG1`	`1`	File identification
`EI_MAG2`	`2`	File identification
`EI_MAG3`	`3`	File identification
`EI_CLASS`	`4`	File class
`EI_DATA`	`5`	Data encoding
`EI_VERSION`	`6`	File version
`EI_OSABI`	`7`	Operating system/ABI identification
`EI_ABIVERSION`	`8`	ABI version
`EI_PAD`	`9`	Start of padding bytes
`EI_NIDENT`	`16`	Size of `e_ident[]`

These indexes access bytes that hold the following values.

`EI_MAG0` to `EI_MAG3`
A file's first 4 bytes hold a ``magic number,'' identifying the file as an ELF object file.

Name	Value	Position
`ELFMAG0`	`0x7f`	`e_ident[EI_MAG0]`
`ELFMAG1`	`'E'`	`e_ident[EI_MAG1]`

ELFMAG2	'L'	e_ident[EI_MAG2]
ELFMAG3	'F'	e_ident[EI_MAG3]

EI_CLASS
: The next byte, e_ident[EI_CLASS], identifies the file's class, or capacity.

Name	Value	Meaning
ELFCLASSNONE	0	Invalid class
ELFCLASS32	1	32-bit objects
ELFCLASS64	2	64-bit objects

The file format is designed to be portable among machines of various sizes, without imposing the sizes of the largest machine on the smallest. The class of the file defines the basic types used by the data structures of the object file container itself. The data contained in object file sections may follow a different programming model. If so, the processor supplement describes the model used.

Class ELFCLASS32 supports machines with 32-bit architectures. It uses the basic types defined in the table labeled ``32-Bit Data Types.''

Class ELFCLASS64 supports machines with 64-bit architectures. It uses the basic types defined in the table labeled ``64-Bit Data Types.''

Other classes will be defined as necessary, with different basic types and sizes for object file data.

EI_DATA
: Byte e_ident[EI_DATA] specifies the encoding of both the data structures used by object file container and data contained in object file sections. The following encodings are currently defined.

Name	Value	Meaning
ELFDATANONE	0	Invalid data encoding
ELFDATA2LSB	1	See below
ELFDATA2MSB	2	See below

Other values are reserved and will be assigned to new encodings as necessary.

 Primarily for the convenience of code that looks at the ELF file at runtime, the ELF data structures are intended to have the same byte order as that of the running program.

EI_VERSION
: Byte e_ident[EI_VERSION] specifies the ELF header version number. Currently, this value must be EV_CURRENT, as explained above for e_version.

EI_OSABI
: Byte e_ident[EI_OSABI] identifies the OS- or ABI-specific ELF extensions used by this file. Some fields in other ELF structures have flags and values that have operating system and/or ABI specific meanings; the interpretation of those fields is determined by the value of this byte. If the object file does not use any extensions, it is recommended that this byte be set to 0. If the value for this byte is 64 through 255, its meaning depends on the value of the e_machine header member. The ABI processor supplement for an architecture can define its own associated set of values for this byte in this range. If the processor supplement does not specify a set of values, one of the following values shall be used, where 0 can also be taken to mean *unspecified*.

Name	Value	Meaning
ELFOSABI_NONE	0	No extensions or unspecified
ELFOSABI_HPUX	1	Hewlett-Packard HP-UX
ELFOSABI_NETBSD	2	NetBSD
ELFOSABI_LINUX	3	Linux
ELFOSABI_SOLARIS	6	Sun Solaris
ELFOSABI_AIX	7	AIX
ELFOSABI_IRIX	8	IRIX
ELFOSABI_FREEBSD	9	FreeBSD
ELFOSABI_TRU64	10	Compaq TRU64 UNIX
ELFOSABI_MODESTO	11	Novell Modesto
ELFOSABI_OPENBSD	12	Open BSD
ELFOSABI_OPENVMS	13	Open VMS
ELFOSABI_NSK	14	Hewlett-Packard Non-Stop Kernel
	64–255	Architecture-specific value range

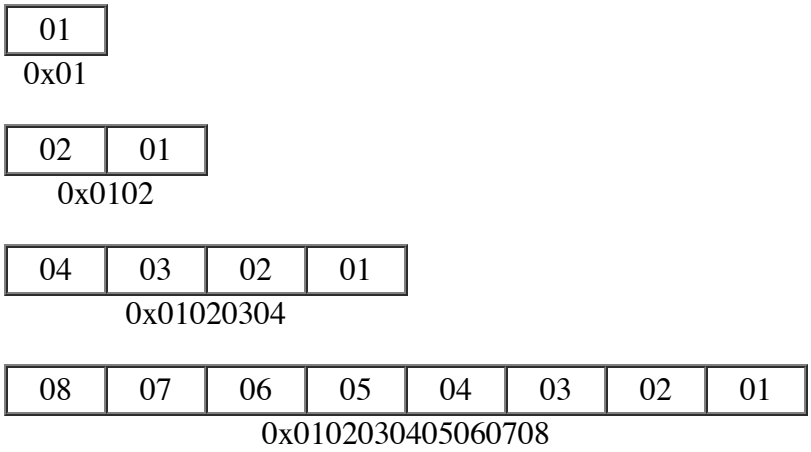
EI_ABIVERSION
: Byte e_ident[EI_ABIVERSION] identifies the version of the ABI to which the object is targeted. This field is used to distinguish among incompatible versions of an ABI. The interpretation of this version number is dependent on the ABI identified by the EI_OSABI field. If no values are specified for the EI_OSABI field by the processor supplement or no version values are specified for the ABI determined by a particular value of the EI_OSABI byte, the value 0 shall be used for the EI_ABIVERSION byte; it indicates *unspecified*.

EI_PAD
: This value marks the beginning of the unused bytes in e_ident. These bytes are reserved and set to zero; programs that read object files should ignore them. The value of EI_PAD will change in the future if currently unused bytes are given meanings.

A file's data encoding specifies how to interpret the basic objects in a file. Class ELFCLASS32 files use objects that occupy 1, 2, and 4 bytes. Class ELFCLASS64 files use objects that occupy 1, 2, 4, and 8 bytes. Under the defined encodings, objects are represented as shown below.

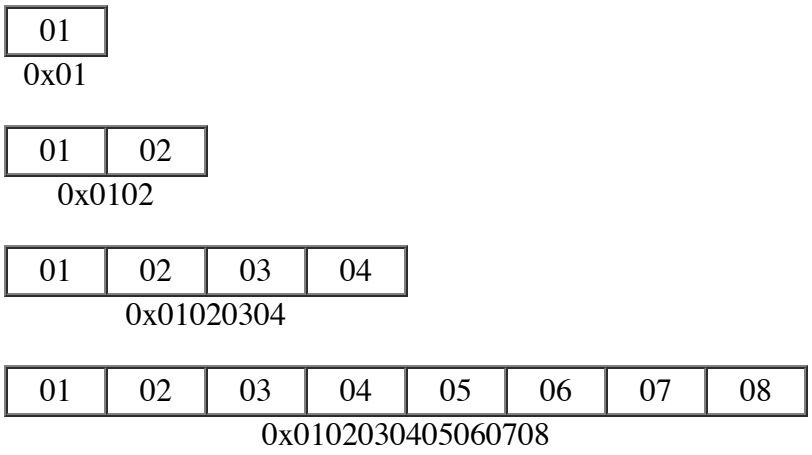
Encoding ELFDATA2LSB specifies 2's complement values, with the least significant byte occupying the lowest address.

Figure 4-5: Data Encoding ELFDATA2LSB, byte address zero on the left



Encoding `ELFDATA2MSB` specifies 2's complement values, with the most significant byte occupying the lowest address.

Figure 4-6: Data Encoding `ELFDATA2MSB`, byte address zero on the left



Machine Information (Processor-Specific)

This section requires processor-specific information. The ABI supplement for the desired processor describes the details.

Previous Contents Next
